

# Mini-Project 1: Cloud Execution Models (Containers and Serverless)

---

## 1. Project Overview

In this mini-project, your group will design and implement one small cloud-native application that combines container-based services and serverless functions. Your application must follow this pattern:

**user submits a form-based request → the system stores the submission → background processing is triggered → the result/status is updated → the user can view the outcome**

The aim is to design a small multi-service cloud system and justify which parts should run as containers and which parts should run as serverless functions.

## 2. Project Description

**Campus Buzz:** Campus information is often time-sensitive, and event notices need to be checked and organised before they are displayed. In this mini-project, your group will build a small cloud application for handling campus event submissions. A user submits a campus event by entering an event title, description, location, date, and organiser name. The system stores the submission, performs an automatic background processing step, and then shows the result to the user. The background check must apply the following rules. First, it must verify that all required fields are present. If any required field is missing, the result must be INCOMPLETE. Second, it must verify that the date field is formatted as YYYY-MM-DD. If the date format is invalid, the result must be NEEDS REVISION. Third, it must check that the description is at least 40 characters long. If the description is too short, the result must be NEEDS REVISION. Fourth, it must assign a category based on keywords in the title or description. If more than one keyword group matches, category assignment must follow this order of precedence: OPPORTUNITY, ACADEMIC, SOCIAL, GENERAL. If the text contains career, internship, or recruitment, the category must be OPPORTUNITY. Otherwise, if it contains a workshop, seminar, or lecture, the category must be ACADEMIC. Otherwise, if it contains club, society, or social, the category must be SOCIAL. Otherwise, the category must be GENERAL. Fifth, it must assign a priority: HIGH for OPPORTUNITY, MEDIUM for ACADEMIC, and NORMAL for SOCIAL or GENERAL. If any required field is missing, the final status must be INCOMPLETE, and no other rule should override that result. Only if all required fields are present should the remaining rules be applied. If the date format is invalid or the description is too short, the final status must be NEEDS REVISION. Otherwise, the final status must be APPROVED. The system must then return one final result: APPROVED, NEEDS REVISION, or INCOMPLETE, according to the rules above. The result shown to the user must include the final status, assigned category, assigned priority, and a short note explaining the result.

This project must be implemented as a hybrid cloud system using both containers and serverless functions. You may customize only small design details, such as the visual style of the interface and the wording of

the short note shown to the user. You may not change the submission type, the required checks, the category and priority logic, the meaning of the result states, the required architecture, the number of required components, the required workflow pattern, or the requirement to use both containers and serverless. The system must not depend on any pre-supplied dataset, benchmark, or external workload. All inputs must be generated through normal use of your own application.

### 3. Required System Components

Your project must contain **exactly six** components: 3 container-based services and 3 serverless functions.

| Component                 | Model      | Required Role                                                        |
|---------------------------|------------|----------------------------------------------------------------------|
| Presentation Service      | Container  | Accept user submissions and show final results                       |
| Workflow Service          | Container  | Create the initial submission record and start background processing |
| Data Service              | Container  | Store and retrieve submission records                                |
| Submission Event Function | Serverless | Turn a new submission event into a processing request                |
| Processing Function       | Serverless | Apply the project rules and compute the result                       |
| Result Update Function    | Serverless | Update the stored record with the computed result                    |

### 4. Required Workflow

Every valid project must support this flow:

**user submits input → workflow creates record → event function starts processing → processing function produces outcome → result function updates system → user views result**

All six required components must participate in the same application.

### 5. Scope Rules

This is a mini-project, not a full-scale platform. Keep the system small and focused.

| Do not include                             | Keep the project focused on                   |
|--------------------------------------------|-----------------------------------------------|
| complex authentication or user accounts    | service decomposition                         |
| large front-end development                | execution-model choice                        |
| ML model training                          | event-driven workflow                         |
| external datasets as the core dependency   | integration between containers and serverless |
| complex distributed infrastructure         |                                               |
| third-party APIs as the core project logic |                                               |

## 6. Use of Generative AI

Generative AI may be used as a support tool, but not as an end-to-end solution generator.

| Allowed uses                                                                    | Not allowed                                                                          |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Understanding the project task, requirements, concepts, documentation, or tools | Submitting AI-generated explanations without understanding them                      |
| Brainstorming ideas, exploring design alternatives, or planning the solution    | Adopting an AI-generated architecture or design with only minimal changes            |
| Generating or improving code, commands, scripts, or configuration files         | Using AI to generate most or all required services/functions as the project solution |
| Explaining errors, outputs, or troubleshooting technical issues                 | Accepting AI-generated fixes without testing and verification                        |
| Testing, validating, reviewing, or improving the solution                       | Using AI to claim validation or review that you did not perform yourself             |
| Writing, refining, or explaining the report, reflection, or design decisions    | Submitting AI-generated report text as your own work with only superficial editing   |

Even when Generative AI is used as a support tool, you are expected to make and justify your own decisions about:

- service decomposition
- component interaction
- execution-model choice
- testing and debugging

## 7. Deliverables

### Group submission

- source code repository link (e.g., GitHub)
- A quick overview of your project (group video report).
  - To explain/demonstrate things that are unclear in your written report.
  - Maximally 4 mins, in English. We do NOT grade your video!
  - Share link using Weiyun (added to the introduction section of your report)

### Individual submission

- Length: **1,200–1,500 words**
- Format: Submit as a **PDF (preferred)** or Word document
- File name: **[ModuleCode][GroupID][StudentID].pdf**
- Anonymization: Do **not include your name, Student ID, or Group ID** in the report.
- Ensure that any screenshots, logs, or links do **not contain identifying information**.

**Note: The individual reports will be peer-reviewed by students later.**